

Alan Medeiros Alves Gonçalves
João Vitor Aguiar de Sousa
Raquel Melo de Queiroz

Relatório de release
Engenharia de Software II

Campina Grande
2025

Março de 2025

Projeto: Marketplace

Histórico de revisões

→ Sempre que este documento for atualizado com novas informações, será registrado a data, a versão e a descrição das atualizações.

Data	Versão	Descrição
10/03/2025	1.0	Estruturação inicial da versão 1.0 do relatório
22/03/2025	1.2	Desenvolvimento da primeira versão do diagrama de caso de uso
26/03/2025	1.3	Desenvolvimento da primeira versão do diagrama de sequência
28/03/2025	1.4	Desenvolvimento da primeira versão do diagrama de classes e adição do relatório de testes

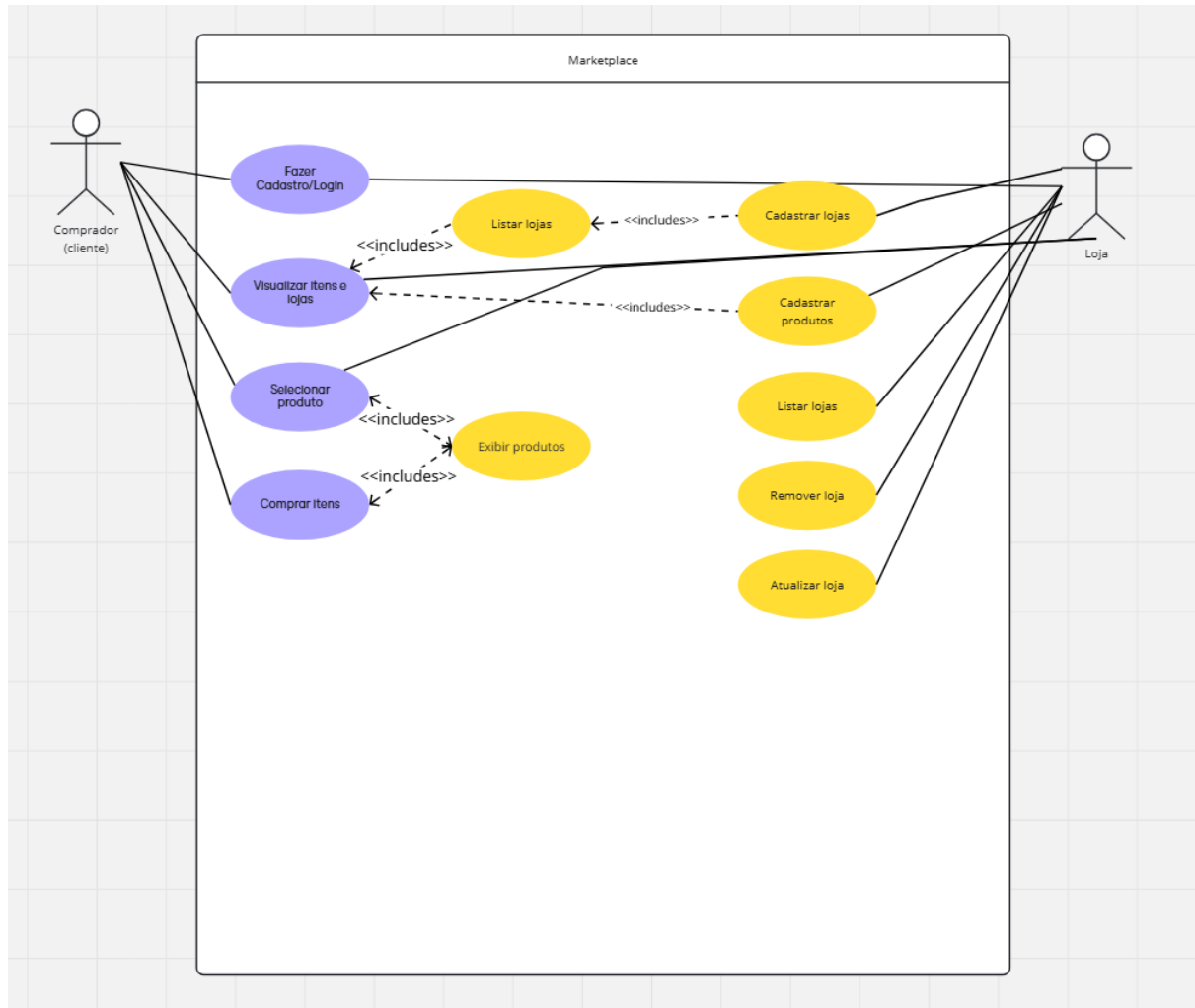
Histórico de revisões e release do documento por release

→ A cada sprint, uma versão deste documento deve ser entregue. No histórico abaixo será demonstrado a versão correspondente a versão final de registros para a sprint correspondente.

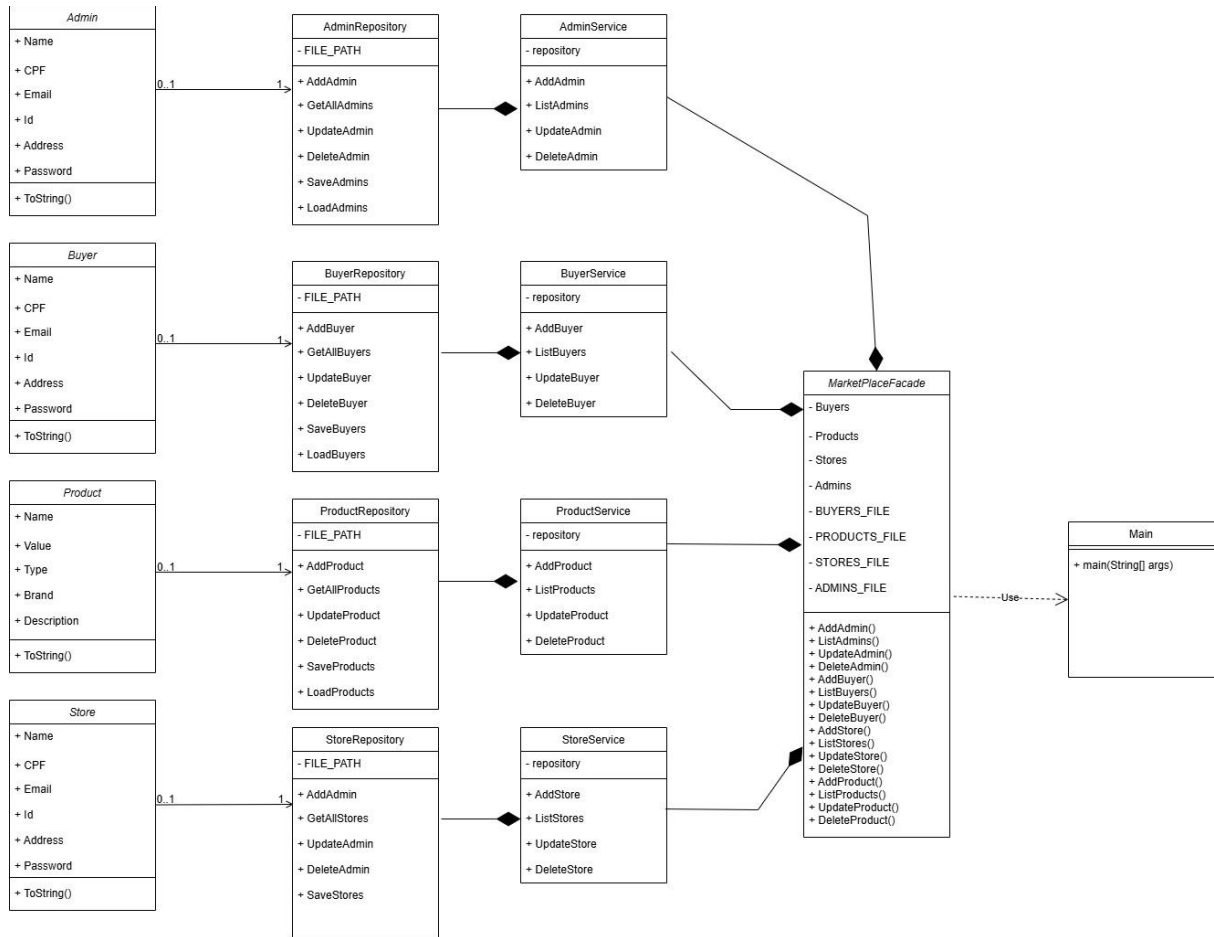
Data	Versão	Descrição
31/03/2025	1.4	Relatório final da release 01

Arquitetura do projeto

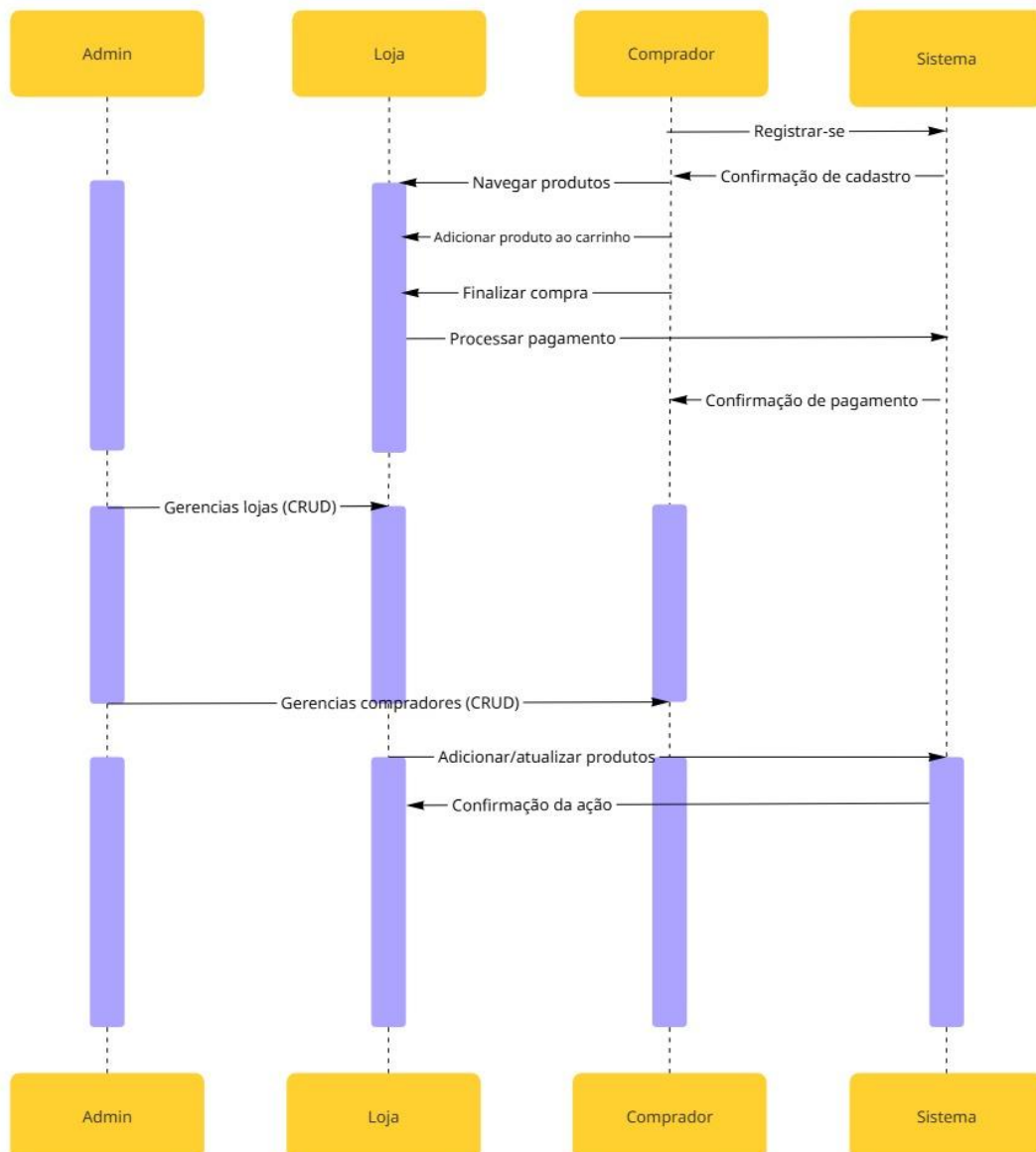
1. Diagrama de caso de uso



2. Diagrama de classes



3. Diagrama de sequência



Relatório de testes

Objetivo:

Os testes realizados visam verificar a funcionalidade das classes de “service” em diversas situações, incluindo adicionar, listar, atualizar e remover. Para isso, os repositórios foram mockados utilizando o Mockito, permitindo que a lógica do serviço fosse testada independentemente da persistência real dos dados e os testes de unidade foram feitos utilizando o JUnit 5.

Relatório - AdminServiceTest

1. Teste: testAddAdmin()

Objetivo:

Verificar se o método addAdmin da classe AdminService adiciona um administrador corretamente.

Descrição do Teste:

- Um administrador foi criado utilizando o método createAdmin.
- O repositório foi mockado para capturar o argumento passado ao método addAdmin usando um ArgumentCaptor.
- O método addAdmin foi chamado no serviço, e foi verificado se o método addAdmin do repositório foi chamado corretamente.

Resultado Esperado:

- O administrador capturado deve ter os mesmos valores que o administrador passado para o serviço.

Resultado Obtido:

- O teste foi bem-sucedido. O nome e email do administrador capturado coincidem com os valores esperados.

2. Teste: testListAdmins()

Objetivo:

Verificar se o método listAdmins da classe AdminService retorna a lista correta de administradores.

Descrição do Teste:

- A lista de administradores foi mockada para retornar dois administradores.
- O método listAdmins foi chamado no serviço, e o retorno foi comparado com a lista esperada.

Resultado Esperado:

- A lista de administradores deve ter tamanho 2 e os nomes dos administradores devem corresponder aos valores esperados.

Resultado Obtido:

- O teste foi bem-sucedido. O tamanho da lista retornada foi 2 e os nomes dos administradores estavam corretos.

3. Teste: testListAdminsEmpty()

Objetivo:

Verificar o comportamento do método listAdmins quando não houver administradores cadastrados.

Descrição do Teste:

- O repositório foi mockado para retornar uma lista vazia de administradores.
- O método listAdmins foi chamado no serviço, e foi verificado se a lista retornada estava vazia.

Resultado Esperado:

- A lista retornada deve estar vazia.

Resultado Obtido:

- O teste foi bem-sucedido. A lista de administradores estava vazia, conforme esperado.

4. Teste: testUpdateAdmin()**Objetivo:**

Verificar se o método updateAdmin da classe AdminService atualiza um administrador corretamente.

Descrição do Teste:

- Um administrador foi criado com o método createAdmin.
- O repositório foi mockado para retornar true quando o método updateAdmin fosse chamado.
- O método updateAdmin foi chamado no serviço, e foi verificado se o método do repositório foi chamado corretamente.

Resultado Esperado:

- O retorno do método updateAdmin deve ser true, indicando que o administrador foi atualizado com sucesso.

Resultado Obtido:

- O teste foi bem-sucedido. O método updateAdmin retornou true, e o método do repositório foi chamado corretamente.

5. Teste: testUpdateAdminNotFound()**Objetivo:**

Verificar o comportamento do método updateAdmin quando o administrador não for encontrado para atualização.

Descrição do Teste:

- Um administrador foi criado com o método createAdmin.

- O repositório foi mockado para retornar false quando o método updateAdmin fosse chamado.
- O método updateAdmin foi chamado no serviço, e foi verificado se o retorno foi false.

Resultado Esperado:

- O retorno do método updateAdmin deve ser false, indicando que o administrador não foi encontrado para atualização.

Resultado Obtido:

- O teste foi bem-sucedido. O método updateAdmin retornou false como esperado.

6. Teste: testRemoveAdmin()**Objetivo:**

Verificar se o método removeAdmin da classe AdminService remove um administrador corretamente.

Descrição do Teste:

- O repositório foi mockado para retornar true quando o método removeAdmin fosse chamado.
- O método removeAdmin foi chamado no serviço, e foi verificado se o método do repositório foi chamado corretamente.

Resultado Esperado:

- O retorno do método removeAdmin deve ser true, indicando que o administrador foi removido com sucesso.

Resultado Obtido:

- O teste foi bem-sucedido. O método removeAdmin retornou true, e o método do repositório foi chamado corretamente.

7. Teste: testRemoveAdminNotFound()

Objetivo:

Verificar o comportamento do método removeAdmin quando o administrador não for encontrado para remoção.

Descrição do Teste:

- O repositório foi mockado para retornar false quando o método removeAdmin fosse chamado.
- O método removeAdmin foi chamado no serviço, e foi verificado se o retorno foi false.

Resultado Esperado:

- O retorno do método removeAdmin deve ser false, indicando que o administrador não foi encontrado para remoção.

Resultado Obtido:

- O teste foi bem-sucedido. O método removeAdmin retornou false como esperado.

8. Teste: testAddAdminThrowsException()

Objetivo:

Verificar o comportamento do método addAdmin quando ocorrer uma exceção.

Descrição do Teste:

- O repositório foi mockado para lançar uma exceção quando o método addAdmin fosse chamado.
- O método addAdmin foi chamado no serviço, e foi verificado se uma exceção foi lançada.

Resultado Esperado:

- Uma exceção do tipo RuntimeException deve ser lançada.

Resultado Obtido:

- O teste foi bem-sucedido. A exceção foi lançada como esperado.

Relatório - BuyerServiceTest

1. Testar Adição de Comprador - testAddBuyer

- **Descrição:** Este teste verifica se a adição de um comprador no repositório ocorre corretamente.
- **Objetivo:** Confirmar que o comprador é adicionado corretamente ao repositório e que os valores capturados são os mesmos passados no método addBuyer.
- **Resultado Esperado:** O comprador deve ser adicionado corretamente ao repositório, e o nome e o e-mail capturados devem ser iguais aos passados.
- **Resultado Obtido:** O teste passou com sucesso. O comprador foi adicionado corretamente e os valores de nome e e-mail estavam corretos.

2. Testar Listagem de Compradores - testListBuyers

- **Descrição:** Este teste verifica se a listagem de compradores no repositório retorna os compradores corretamente.
- **Objetivo:** Verificar que a listagem de compradores retorna uma lista correta com os nomes dos compradores.
- **Resultado Esperado:** A lista deve conter dois compradores, com os nomes corretamente listados.
- **Resultado Obtido:** O teste passou com sucesso. A lista continha dois compradores e os nomes estavam corretos.

3. Testar Listagem de Compradores Vazia - testListBuyersEmpty

- **Descrição:** Este teste verifica se a listagem de compradores retorna uma lista vazia quando não há compradores no repositório.
- **Objetivo:** Garantir que, quando não houver compradores, a lista retornada pelo método listBuyers esteja vazia.
- **Resultado Esperado:** O método deve retornar uma lista vazia, pois não há compradores.
- **Resultado Obtido:** O teste passou com sucesso. A lista retornada estava vazia.

4. Testar Atualização de Comprador - testUpdateBuyer

- **Descrição:** Este teste verifica se o comprador é atualizado corretamente quando encontrado no repositório.
- **Objetivo:** Confirmar que, quando o comprador é encontrado, o método updateBuyer realiza a atualização corretamente.
- **Resultado Esperado:** O comprador deve ser atualizado corretamente, e o método de atualização deve ser chamado uma vez.
- **Resultado Obtido:** O teste passou com sucesso. O comprador foi atualizado corretamente, e o método foi chamado uma vez.

5. Testar Atualização de Comprador Não Encontrado - testUpdateBuyerNotFound

- **Descrição:** Este teste verifica se a atualização de um comprador retorna false quando o comprador não é encontrado no repositório.
- **Objetivo:** Garantir que, quando o comprador não for encontrado, a operação de atualização falhe e retorne false.
- **Resultado Esperado:** O comprador não deve ser atualizado, e o resultado deve ser false.

- **Resultado Obtido:** O teste passou com sucesso. O resultado foi false, indicando que o comprador não foi encontrado.

6. Testar Remoção de Comprador - testRemoveBuyer

- **Descrição:** Este teste verifica se a remoção de um comprador funciona corretamente.
- **Objetivo:** Confirmar que o comprador é removido corretamente quando encontrado no repositório.
- **Resultado Esperado:** O comprador deve ser removido corretamente, e o método de remoção deve ser chamado uma vez.
- **Resultado Obtido:** O teste passou com sucesso. O comprador foi removido corretamente, e o método foi chamado uma vez.

7. Testar Remoção de Comprador Não Encontrado - testRemoveBuyerNotFound

- **Descrição:** Este teste verifica se a remoção de um comprador retorna false quando o comprador não é encontrado no repositório.
- **Objetivo:** Garantir que, quando o comprador não for encontrado, a remoção falhe e retorne false.
- **Resultado Esperado:** O comprador não deve ser removido, e o resultado deve ser false.
- **Resultado Obtido:** O teste passou com sucesso. O resultado foi false, indicando que o comprador não foi encontrado.

8. Testar Exceção ao Adicionar Comprador - testAddBuyerThrowsException

- **Descrição:** Este teste verifica se uma exceção é lançada corretamente ao tentar adicionar um comprador quando o repositório gera uma exceção.

- **Objetivo:** Garantir que o serviço lance uma exceção quando o repositório falha ao adicionar um comprador.
- **Resultado Esperado:** Uma exceção deve ser lançada quando ocorre um erro ao adicionar o comprador.
- **Resultado Obtido:** O teste passou com sucesso. A exceção foi lançada corretamente.

Relatório - ProductServiceTest

1. Teste: testAddProduct

- **Objetivo:** Verificar se o método addProduct da classe ProductService adiciona um produto corretamente.
- **Descrição do Teste:** Foi criado um produto mockado, e o método addProduct foi chamado. O repositório foi mockado para capturar o produto inserido.
- **Resultado Esperado:** O produto adicionado deve ser igual ao produto capturado no repositório.
- **Resultado Obtido:** O teste foi bem-sucedido. O produto foi adicionado corretamente, e o método do repositório foi chamado uma vez, com os dados corretos.

2. Teste: testListProducts

- **Objetivo:** Verificar se o método listProducts da classe ProductService retorna a lista de produtos corretamente.
- **Descrição do Teste:** Foram criados dois produtos mockados. O repositório foi mockado para retornar esses dois produtos. O método listProducts foi chamado, e a lista de produtos foi verificada.

- **Resultado Esperado:**A lista de produtos retornada deve conter dois produtos, e os nomes desses produtos devem ser "Notebook Acer" e "Notebook Multilaser".
- **Resultado Obtido:**O teste foi bem-sucedido. A lista retornada contém dois produtos e os nomes estão corretos.

3. Teste: testListProductsEmpty

- **Objetivo:**Verificar se o método listProducts retorna uma lista vazia quando não há produtos.
- **Descrição do Teste:**O repositório foi mockado para retornar uma lista vazia. O método listProducts foi chamado, e a lista foi verificada.
- **Resultado Esperado:**A lista retornada deve ser vazia.
- **Resultado Obtido:**O teste foi bem-sucedido. A lista retornada estava vazia, conforme esperado.

4. Teste: testUpdateProduct_Success

- **Objetivo:**Verificar se o método updateProduct da classe ProductService atualiza um produto corretamente.
- **Descrição do Teste:**Um produto mockado foi criado e o repositório foi mockado para retornar true quando o método updateProduct fosse chamado. O método updateProduct foi chamado, e o resultado foi verificado.
- **Resultado Esperado:**O método updateProduct deve retornar true, indicando que o produto foi atualizado com sucesso.

- **Resultado Obtido:**O teste foi bem-sucedido. O método retornou true, e o repositório foi chamado corretamente.

5. Teste: testUpdateProduct_NotFound

- **Objetivo:**Verificar se o método updateProduct da classe ProductService retorna false quando o produto não for encontrado.
- **Descrição do Teste:** Foi criado um produto mockado e o repositório foi mockado para retornar false quando o método updateProduct fosse chamado. O método updateProduct foi chamado, e o resultado foi verificado.
- **Resultado Esperado:** O método updateProduct deve retornar false, indicando que o produto não foi encontrado para atualização.
- **Resultado Obtido:**O teste foi bem-sucedido. O método retornou false, como esperado, indicando que o produto não foi encontrado.

6. Teste: testRemoveProduct_Success

- **Objetivo:**Verificar se o método removeProduct da classe ProductService remove um produto corretamente.
- **Descrição do Teste:**O repositório foi mockado para retornar true quando o método removeProduct fosse chamado. O método removeProduct foi chamado, e o resultado foi verificado.
- **Resultado Esperado:**O método removeProduct deve retornar true, indicando que o produto foi removido com sucesso.
- **Resultado Obtido:**O teste foi bem-sucedido. O método retornou true, e o repositório foi chamado corretamente.

7. Teste: testRemoveProduct_NotFound

- **Objetivo:** Verificar se o método removeProduct da classe ProductService retorna false quando o produto não for encontrado.
- **Descrição do Teste:** O repositório foi mockado para retornar false quando o método removeProduct fosse chamado. O método removeProduct foi chamado, e o resultado foi verificado.
- **Resultado Esperado:** O método removeProduct deve retornar false, indicando que o produto não foi encontrado para remoção.
- **Resultado Obtido:** O teste foi bem-sucedido. O método retornou false, como esperado, indicando que o produto não foi encontrado.

8. Teste: testAddProductThrowsException

- **Objetivo:** Verificar se o método addProduct da classe ProductService lança uma exceção corretamente quando ocorre um erro no repositório.
- **Descrição do Teste:** Foi criado um produto mockado, e o repositório foi mockado para lançar uma exceção quando o método addProduct fosse chamado. O método addProduct foi chamado, e a exceção foi verificada.
- **Resultado Esperado:** Uma exceção de tipo RuntimeException deve ser lançada ao adicionar o produto.
- **Resultado Obtido:** O teste foi bem-sucedido. A exceção foi lançada conforme esperado, e o teste passou.

9. Teste: testUpdateProductThrowsException

- **Objetivo:** Verificar se o método updateProduct da classe ProductService lança uma exceção corretamente quando ocorre um erro no repositório.

- **Descrição do Teste:**Foi criado um produto mockado, e o repositório foi mockado para lançar uma exceção quando o método updateProduct fosse chamado. O método updateProduct foi chamado, e a exceção foi verificada.
- **Resultado Esperado:**Uma exceção de tipo RuntimeException deve ser lançada ao atualizar o produto.
- **Resultado Obtido:**O teste foi bem-sucedido. A exceção foi lançada conforme esperado, e o teste passou.

Relatório - StoreServiceTest

1. Teste: testAddStore

- **Objetivo:**Verificar se o método addStore da classe StoreService adiciona uma loja corretamente.
- **Descrição do Teste:**Foi criada uma loja mockada com os dados necessários e o método addStore foi chamado. O repositório foi mockado para capturar o produto inserido. O método foi verificado para garantir que o repositório foi chamado corretamente.
- **Resultado Esperado:**O método addStore deve capturar a loja corretamente, e os dados de nome e e-mail devem corresponder aos da loja adicionada.
- **Resultado Obtido:**O teste foi bem-sucedido. O método foi chamado uma vez e os dados do repositório estavam corretos.

2. Teste: testListStores

- **Objetivo:**Verificar se o método listStores retorna a lista de lojas corretamente.

- **Descrição do Teste:**Duas lojas mockadas foram criadas e o repositório foi mockado para retornar essas lojas. O método listStores foi chamado, e a lista foi verificada para garantir que ela contenha as lojas esperadas.
- **Resultado Esperado:**A lista de lojas deve conter duas lojas, e seus dados (nome, e-mail e CNPJ) devem ser os corretos.
- **Resultado Obtido:**O teste foi bem-sucedido. A lista de lojas retornada contém duas lojas e os dados estavam corretos.

3. Teste: testListStoresEmpty

- **Objetivo:**Verificar se o método listStores retorna uma lista vazia quando não há lojas.
- **Descrição do Teste:**O repositório foi mockado para retornar uma lista vazia. O método listStores foi chamado e a lista resultante foi verificada.
- **Resultado Esperado:**A lista retornada deve ser vazia.
- **Resultado Obtido:**O teste foi bem-sucedido. A lista estava vazia como esperado.

4. Teste: testUpdateStore_Success

- **Objetivo:**Verificar se o método updateStore atualiza uma loja corretamente.
- **Descrição do Teste:**Uma loja foi criada com um ID fictício para simular a existência no banco de dados. O repositório foi mockado para retornar true ao atualizar a loja. O método updateStore foi chamado e o resultado foi verificado.
- **Resultado Esperado:**O método updateStore deve retornar true, indicando que a loja foi atualizada com sucesso.

- **Resultado Obtido:**O teste foi bem-sucedido. O método retornou true, e o repositório foi chamado corretamente.

5. Teste: testUpdateStore_NotFound

- **Objetivo:**Verificar se o método updateStore retorna false quando a loja não é encontrada.
- **Descrição do Teste:**Uma loja com um ID fictício foi criada, e o repositório foi mockado para retornar false ao tentar atualizar essa loja. O método updateStore foi chamado e o resultado foi verificado.
- **Resultado Esperado:**O método updateStore deve retornar false, indicando que a loja não foi encontrada e não foi atualizada.
- **Resultado Obtido:**O teste foi bem-sucedido. O método retornou false, como esperado, indicando que a loja não foi encontrada.

6. Teste: testRemoveStore_Success

- **Objetivo:**Verificar se o método removeStore remove uma loja corretamente.
- **Descrição do Teste:**Foi criado um ID de loja, e o repositório foi mockado para retornar true ao tentar remover essa loja. O método removeStore foi chamado e o resultado foi verificado.
- **Resultado Esperado:**O método removeStore deve retornar true, indicando que a loja foi removida com sucesso.
- **Resultado Obtido:**O teste foi bem-sucedido. O método retornou true, e o repositório foi chamado corretamente.

7. Teste: testRemoveStore_NotFound

- **Objetivo:**Verificar se o método removeStore retorna false quando a loja não é encontrada.

- **Descrição do Teste:**Foi criado um ID de loja fictício, e o repositório foi mockado para retornar false ao tentar remover essa loja. O método removeStore foi chamado e o resultado foi verificado.
- **Resultado Esperado:**O método removeStore deve retornar false, indicando que a loja não foi encontrada e não foi removida.
- **Resultado Obtido:**O teste foi bem-sucedido. O método retornou false, como esperado, indicando que a loja não foi encontrada.

8. Teste: testAddStoreThrowsException

- **Objetivo:**Verificar se o método addStore lança uma exceção corretamente quando ocorre um erro no repositório.
- **Descrição do Teste:**Foi criada uma loja mockada e o repositório foi mockado para lançar uma exceção ao tentar adicionar a loja. O método addStore foi chamado e a exceção foi verificada.
- **Resultado Esperado:**Uma exceção de tipo RuntimeException deve ser lançada ao tentar adicionar a loja.
- **Resultado Obtido:**O teste foi bem-sucedido. A exceção foi lançada corretamente, como esperado.

9. Teste: testUpdateStoreThrowsException

- **Objetivo:**Verificar se o método updateStore lança uma exceção corretamente quando ocorre um erro no repositório.
- **Descrição do Teste:**Foi criada uma loja mockada e o repositório foi mockado para lançar uma exceção ao tentar atualizar a loja. O método updateStore foi chamado e a exceção foi verificada.

- **Resultado Esperado:** Uma exceção de tipo RuntimeException deve ser lançada ao tentar atualizar a loja.
- **Resultado Obtido:** O teste foi bem-sucedido. A exceção foi lançada corretamente, como esperado.

Conclusão:

Todos os testes realizados passaram com sucesso, verificando de forma eficaz as principais funcionalidades das classes de service(CRUD 's). Através do uso do Mockito, foi possível simular o comportamento do repositório e testar as interações do serviço de forma isolada. Os casos de sucesso e falha foram bem cobertos, garantindo que a lógica da aplicação estivesse correta para diferentes cenários.